

Project: Dinamiche replicative

evoluzione strategica, competizione sociale e dinamiche collettive

1. Obiettivi della dispensa

Questa dispensa introduce le dinamiche replicative come caso di studio per un corso di metodi computazionali per modelli stocastici e dinamiche collettive.

Gli obiettivi sono cinque:

1. derivare la replicator equation a partire da interazioni tra strategie;
2. studiare il caso a due strategie e la sua riduzione a una equazione differenziale in una variabile;
3. analizzare punti fissi, stabilita' locale e interpretazione dinamica;
4. estendere il formalismo al caso di piu' strategie;
5. collegare il modello continuo a versioni stocastiche a popolazione finita, come le dinamiche di tipo Moran.

Dal punto di vista del corso, questo modello e' particolarmente interessante perche' si colloca al confine tra teoria dei giochi evolutiva, sociologia matematica, apprendimento collettivo e dinamiche di popolazione.

2. Motivazione generale

Le dinamiche replicative descrivono come cambia nel tempo la frequenza di diverse strategie, comportamenti o tipi all'interno di una popolazione.

Esempi naturali sono:

- diffusione di norme sociali;
- competizione tra opinioni;
- diffusione di comportamenti cooperativi o opportunistici;
- selezione di strategie in contesti economici;
- evoluzione culturale;
- apprendimento sociale per imitazione.

L'idea di fondo e' semplice: una strategia cresce se ottiene una performance superiore alla media della popolazione, e diminuisce se ottiene una performance inferiore alla media.

In questo senso, il modello traduce in forma matematica una regola di selezione relativa.

3. Popolazione e strategie

Supponiamo che una popolazione possa adottare n strategie diverse. Indichiamo con

$$x_i(t)$$

la frazione della popolazione che al tempo t utilizza la strategia i , con

$$x_i(t) \geq 0, \quad \sum_{i=1}^n x_i(t) = 1.$$

Il vettore

$$x(t) = (x_1(t), \dots, x_n(t))$$

appartiene quindi al semplice delle frequenze.

Interpretazione:

- $x_i(t)$ non e' un numero assoluto di individui, ma una quota relativa;
- la dinamica si svolge nello spazio delle distribuzioni di frequenza.

4. Matrice dei payoff

Supponiamo che l'interazione tra strategie sia descritta da una matrice dei payoff

$$A = (a_{ij}),$$

dove a_{ij} rappresenta il payoff ottenuto da un individuo che usa la strategia i quando incontra un individuo che usa la strategia j .

Se la popolazione e' ben mescolata, il payoff atteso della strategia i nella popolazione x e'

$$f_i(x) = \sum_{j=1}^n a_{ij}x_j.$$

In forma vettoriale:

$$f(x) = Ax.$$

La fitness media della popolazione e'

$$\bar{f}(x) = \sum_{i=1}^n x_i f_i(x) = x^T Ax.$$

Questa quantita' rappresenta il payoff medio atteso di un individuo scelto a caso nella popolazione.

5. Derivazione della replicator equation

L'idea centrale e' che la frequenza di una strategia cresce se il suo payoff supera la media e decresce nel caso opposto.

Una forma naturale di questa idea e'

$$\dot{x}_i \propto x_i (f_i(x) - \bar{f}(x)).$$

La presenza del fattore x_i e' importante:

- se una strategia e' assente, non puo' crescere spontaneamente nella dinamica replicativa pura;
- la variazione relativa dipende dalla sua abbondanza attuale.

Scegliendo la costante di proporzionalita' uguale a 1, si ottiene la replicator equation:

$$\dot{x}_i = x_i (f_i(x) - \bar{f}(x)), \quad i = 1, \dots, n.$$

Sostituendo $f_i(x) = (Ax)_i$ e $\bar{f}(x) = x^T Ax$, la forma standard e'

$$\dot{x}_i = x_i [(Ax)_i - x^T Ax].$$

Questa e' l'equazione fondamentale della dispensa.

6. Proprieta' di base

6.1 Conservazione del semplice

Se inizialmente

$$\sum_{i=1}^n x_i = 1,$$

allora questa proprieta' resta vera nel tempo.

Infatti,

$$\sum_{i=1}^n \dot{x}_i = \sum_{i=1}^n x_i (f_i - \bar{f}) = \sum_{i=1}^n x_i f_i - \bar{f} \sum_{i=1}^n x_i = \bar{f} - \bar{f} = 0.$$

Quindi il simpleso e' invariante.

6.2 Significato dinamico

La dinamica non dipende dal payoff assoluto, ma dalla differenza tra payoff individuale e payoff medio.

Questo significa che il modello e' intrinsecamente relativo:

- conta il confronto con la media della popolazione;
- non conta soltanto il livello assoluto della performance.

7. Caso a due strategie

7.1 Riduzione a una variabile

Se ci sono solo due strategie, basta conoscere la frequenza della prima:

$$x = x_1, \quad x_2 = 1 - x.$$

La dinamica si riduce allora a una sola equazione.

Supponiamo che la matrice dei payoff sia

$$A = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix}.$$

I payoff attesi sono

$$f_1(x) = a_{11}x + a_{12}(1 - x),$$

$$f_2(x) = a_{21}x + a_{22}(1 - x).$$

La replicator equation diventa

$$\dot{x} = x(1-x)(f_1(x) - f_2(x)).$$

Questa forma e' molto importante. Mostra che la dinamica si annulla sempre ai bordi

$$x = 0, \quad x = 1,$$

e puo' avere punti fissi interni quando

$$f_1(x) = f_2(x).$$

7.2 Punto fisso interno

Imponendo

$$a_{11}x + a_{12}(1-x) = a_{21}x + a_{22}(1-x),$$

si ottiene

$$x^* = \frac{a_{22} - a_{12}}{a_{11} - a_{12} - a_{21} + a_{22}},$$

quando il denominatore e' non nullo.

Questo punto fisso e' significativo solo se appartiene all'intervallo $[0, 1]$.

7.3 Analisi qualitativa

Il caso a due strategie e' particolarmente utile per l'analisi qualitativa.

La dinamica dipende dal segno di

$$f_1(x) - f_2(x).$$

Se questa differenza e' positiva, allora

$$\dot{x} > 0$$

nell'interno dell'intervallo, e la strategia 1 tende a crescere.

Se la differenza e' negativa, allora la frequenza della strategia 1 tende a diminuire.

Dal punto di vista grafico, si puo' fare una analisi di linea di fase sull'intervallo $[0, 1]$.

8. Interpretazione dei diversi casi a due strategie

Il caso a due strategie permette di distinguere diversi regimi dinamici.

8.1 Dominanza della strategia 1

Se

$$f_1(x) > f_2(x) \quad \text{per ogni } x \in (0, 1),$$

allora la strategia 1 invade e la popolazione converge verso

$$x = 1.$$

8.2 Dominanza della strategia 2

Se

$$f_1(x) < f_2(x) \quad \text{per ogni } x \in (0, 1),$$

allora la popolazione converge verso

$$x = 0.$$

8.3 Coordinamento

Se uno dei due stati puri e' stabile e l'altro pure, con un punto fisso interno instabile, allora la dinamica e' di coordinamento.

In questo caso le condizioni iniziali determinano a quale equilibrio il sistema converge.

8.4 Convivenza

Se il punto fisso interno e' stabile e i due bordi sono instabili, allora la dinamica tende a una miscela stabile delle due strategie.

Questo e' uno dei casi piu' interessanti per interpretazioni sociali, perche' rappresenta la persistenza di pluralismo comportamentale.

9. Stabilita' locale

9.1 Idea generale

Un punto fisso x^* e' stabile se piccole perturbazioni attorno ad esso si riassorbono nel tempo.

Nel caso unidimensionale, basta guardare il segno della derivata del campo dinamico

$$F(x) = x(1-x)(f_1(x) - f_2(x)).$$

Se

$$F'(x^*) < 0,$$

il punto fisso e' localmente stabile.

Se

$$F'(x^*) > 0,$$

e' instabile.

9.2 Bordi del semplice

Anche gli stati puri

$$x = 0, \quad x = 1$$

sono punti fissi. La loro stabilita' dipende da whether una piccola invasione della strategia alternativa cresce o si estingue.

Questo lega direttamente la replicator equation al concetto di invasibilita' evolutiva.

10. Caso a piu' strategie

10.1 Forma generale

Nel caso $n > 2$, la replicator equation resta

$$\dot{x}_i = x_i [(Ax)_i - x^T Ax], \quad i = 1, \dots, n.$$

La dinamica si svolge sul simpleso

$$\Delta_n = \left\{ x \in \mathbb{R}^n : x_i \geq 0, \sum_i x_i = 1 \right\}.$$

Per $n = 3$, il simpleso e' un triangolo, e quindi si presta molto bene a rappresentazioni geometriche e numeriche.

10.2 Punti fissi interni

Un punto fisso interno x^* con tutte le componenti positive deve soddisfare

$$(Ax^*)_1 = (Ax^*)_2 = \dots = (Ax^*)_n = \bar{f}(x^*).$$

Cioe', in un equilibrio interno tutte le strategie presenti con frequenza positiva devono avere lo stesso payoff.

Questa e' una condizione molto importante e molto intuitiva: se una strategia presente avesse payoff maggiore, crescerebbe; se avesse payoff minore, diminuirebbe.

10.3 Lettura geometrica

Nel caso tridimensionale, gli stati puri corrispondono ai vertici del triangolo. Gli stati misti corrispondono a punti interni o lungo i lati.

Questo rende possibile una ricca analisi visuale:

- flussi verso i vertici;
- attrattori interni;
- cicli o quasi-cicli in alcuni casi speciali;
- separatrici tra bacini di attrazione.

11. Connessione con la teoria dei giochi evolutiva

Le dinamiche replicative sono strettamente collegate alla teoria dei giochi evolutiva.

Le strategie non sono necessariamente il risultato di una deliberazione perfettamente razionale. Possono essere interpretate come:

- comportamenti imitati;
- norme sociali;
- tratti culturali;
- pratiche che si diffondono se hanno successo relativo;

- routine organizzative.

In questo senso, la replicator equation e' uno strumento estremamente flessibile per passare dall'interazione strategica alla dinamica delle frequenze.

12. Interpretazione sociologica

Questo modello non va letto solo in chiave biologica o evolutiva in senso stretto. Può essere reinterpretato in molti contesti sociali.

Opinioni

Le strategie sono opinioni o orientamenti valoriali.

Comportamenti

Le strategie rappresentano pratiche sociali, come cooperare, conformarsi o deviare.

Norme

Le strategie sono regole implicite o convenzioni che si rafforzano se funzionano meglio della media.

Identita' collettive

Le strategie possono rappresentare modelli di appartenenza o stili di interazione.

Da questo punto di vista, le dinamiche replicative sono particolarmente adatte a una lettura interdisciplinare tra matematica, economia, sociologia e scienze cognitive.

13. Collegamento con popolazioni finite

13.1 Limite del modello continuo

La replicator equation e' una dinamica continua su frequenze. Questo implica:

- popolazione molto grande;
- mescolamento omogeneo;
- assenza di rumore demografico.

In una popolazione finita, invece, le frequenze cambiano per eventi discreti e stocastici.

13.2 Dinamiche di tipo Moran

Una controparte naturale e' il processo di Moran.

L'idea di base e':

1. si sceglie un individuo da riprodurre con probabilita' proporzionale alla fitness;
2. si sceglie un individuo da rimpiazzare;
3. la composizione della popolazione cambia di una unita'.

In grande popolazione e con opportuni limiti di scala, queste dinamiche stocastiche portano a una descrizione media vicina alla replicator equation.

13.3 Perche' questo e' importante nel corso

Questo passaggio e' didatticamente molto ricco, perche' consente di distinguere tra:

- dinamica deterministica media;
- dinamica stocastica a popolazione finita;
- effetti di rumore, estinzione casuale e fluttuazioni.

E' esattamente uno dei punti forti del modulo.

14. Pseudocodice del caso a due strategie

Consideriamo la dinamica

$$\dot{x} = x(1-x)(f_1(x) - f_2(x)).$$

Per una simulazione numerica semplice si puo' usare Euler in tempo discreto.

Input

- matrice dei payoff A
- valore iniziale x_0
- passo temporale Δt
- numero di iterazioni T

Pseudocodice

1. inizializza $x = x_0$
2. per $t = 1, \dots, T$:
 - calcola

$$f_1(x), \quad f_2(x)$$

- calcola

$$\dot{x} = x(1-x)(f_1(x) - f_2(x))$$

- aggiorna

$$x \leftarrow x + \Delta t \dot{x}$$

- se necessario, tronca x all'intervallo $[0, 1]$
3. salva la traiettoria

Questo e' il modo piu' semplice per passare dalla teoria alla simulazione.

15. Pseudocodice del caso a tre strategie

Per $n = 3$ si lavora con il vettore

$$x = (x_1, x_2, x_3).$$

Input

- matrice A
- stato iniziale $x(0)$
- passo temporale Δt
- numero di iterazioni T

Pseudocodice

1. inizializza x
2. per ogni tempo:

- calcola

$$f = Ax$$

- calcola la fitness media

$$\bar{f} = x^T Ax$$

- per ogni strategia i :

$$\dot{x}_i = x_i(f_i - \bar{f})$$

- aggiorna

$$x_i \leftarrow x_i + \Delta t \dot{x}_i$$

- rinormalizza se necessario

3. salva la traiettoria nel simpleso

Questo schema e' ideale per costruire diagrammi di fase numerici.

16. Schema del laboratorio

16.1 Laboratorio 1 - Due strategie

Obiettivo

Studiare la linea di fase della replicator equation nel caso bidimensionale.

Attivita'

1. scegliere una matrice 2×2 ;
2. derivare $f_1(x)$ e $f_2(x)$;
3. trovare i punti fissi;
4. classificarne la stabilita';
5. simulare numericamente la traiettoria per diverse condizioni iniziali.

Domande guida

- esiste un punto fisso interno?
- i bordi sono stabili o instabili?
- la dinamica e' di dominanza, coordinamento o convivenza?

16.2 Laboratorio 2 - Tre strategie

Obiettivo

Studiare la dinamica sul simpleso.

Attivita'

1. scegliere una matrice 3×3 ;
2. simulare le traiettorie da punti iniziali diversi;
3. rappresentare i flussi sul triangolo;
4. identificare attrattori, separatrici o cicli.

Domande guida

- esiste un equilibrio interno?
- il sistema converge ai vertici o a uno stato misto?
- quali regioni iniziali portano a esiti diversi?

16.3 Laboratorio 3 - Confronto con popolazione finita

Obiettivo

Confrontare la dinamica replicativa continua con una simulazione discreta di tipo Moran.

Attività

1. fissare una popolazione finita N ;
2. simulare nascita e sostituzione con probabilità proporzionali alla fitness;
3. osservare la traiettoria delle frequenze;
4. confrontare la media di molte simulazioni con la traiettoria della replicator equation.

Domande guida

- quanto sono grandi le fluttuazioni finite?
- la media su molte simulazioni segue la dinamica deterministica?
- quali esiti compaiono solo per via del rumore demografico?

17. Perché questo è un buon case study

Questo modello è particolarmente forte per il corso per almeno quattro ragioni.

Primo, ha una interpretazione matematica chiara e pulita.

Secondo, è facile da simulare numericamente.

Terzo, si presta bene a una lettura interdisciplinare in termini di comportamenti, norme e competizione sociale.

Quarto, collega in modo naturale:

- dinamica continua deterministica;
- analisi di stabilità;
- simulazione numerica;
- versioni stocastiche a popolazione finita.

Per questo motivo costituisce uno dei case study più completi dell'intero corso.

18. Conclusione

Le dinamiche replicative formalizzano una idea semplice e potente: strategie con performance superiore alla media tendono a diffondersi, mentre strategie meno performanti tendono a scomparire. A partire da questa idea si ottiene una classe

molto ricca di modelli capaci di descrivere evoluzione strategica, competizione sociale, selezione culturale e apprendimento collettivo.

Dal punto di vista didattico, il modello e' particolarmente utile perche' consente di passare in modo naturale:

- dal caso a due strategie alla dinamica multidimensionale;
- dall'analisi qualitativa alla simulazione numerica;
- dal modello continuo ai processi stocastici a popolazione finita.

19. Bibliografia minima

1. Taylor, P. D., and Jonker, L. B. (1978). Evolutionarily Stable Strategies and Game Dynamics. *Mathematical Biosciences*, 40, 145-156.
 2. Hofbauer, J., and Sigmund, K. (1998). *Evolutionary Games and Population Dynamics*. Cambridge University Press.
 3. Weibull, J. W. (1995). *Evolutionary Game Theory*. MIT Press.
 4. Nowak, M. A. (2006). *Evolutionary Dynamics*. Harvard University Press.
 5. Sandholm, W. H. (2010). *Population Games and Evolutionary Dynamics*. MIT Press.
-

Appendice A. Implementazione in Python: struttura del codice

Questa appendice propone una struttura semplice per implementare in Python i modelli discussi nella dispensa:

1. dinamica replicativa a due strategie;
2. dinamica replicativa a tre strategie;
3. mini-simulazione stocastica a popolazione finita di tipo Moran.

L'obiettivo non e' costruire un codice ottimizzato, ma fornire una guida leggibile:

- per chi conosce Python, il codice e' quasi direttamente implementabile;
- per chi usa altri linguaggi, la struttura puo' essere letta come pseudocodice operativo.

Per questo motivo il codice e' volutamente elementare:

- poche librerie;
- liste e cicli espliciti;
- poche astrazioni;
- funzioni corte;
- commenti minimi ma chiari.

A.1 Librerie minime

Per tutto quello che segue bastano:

```
import random
import statistics
import matplotlib.pyplot as plt
```

Quindi:

- `random` serve per scelte casuali e simulazioni stocastiche;
- `statistics` serve per medie semplici;
- `matplotlib.pyplot` serve per grafici.

Non e' necessario usare `numpy` in una prima implementazione.

A.2 Rappresentare la matrice dei payoff

Una matrice dei payoff puo' essere rappresentata come lista di liste.

Per esempio, nel caso a due strategie:

```
A = [
    [3.0, 1.0],
    [2.0, 2.0]
]
```

Nel caso a tre strategie:

```
A = [
    [0.0, 2.0, 1.0],
    [1.0, 0.0, 2.0],
    [2.0, 1.0, 0.0]
]
```

In generale:

- `A[i][j]` e' il payoff della strategia `i` contro la strategia `j`.

A.3 Funzioni di base per il caso generale

A.3.1 Prodotto matrice-vettore

Data una matrice A e un vettore di frequenze x , il vettore dei payoff attesi e'

$$f(x) = Ax.$$

In Python:

```

def matrix_vector_product(A, x):
    result = []

    for i in range(len(A)):
        total = 0.0
        for j in range(len(x)):
            total += A[i][j] * x[j]
        result.append(total)

    return result

```

A.3.2 Fitness media

La fitness media e'

$$\bar{f}(x) = x^T A x.$$

In Python:

```

def average_fitness(A, x):
    payoffs = matrix_vector_product(A, x)

    total = 0.0
    for i in range(len(x)):
        total += x[i] * payoffs[i]

    return total

```

A.3.3 Campo replicativo

La replicator equation generale e'

$$\dot{x}_i = x_i [(Ax)_i - x^T A x].$$

In Python:

```

def replicator_field(A, x):
    payoffs = matrix_vector_product(A, x)
    avg_fit = average_fitness(A, x)

    dx = []

    for i in range(len(x)):
        value = x[i] * (payoffs[i] - avg_fit)
        dx.append(value)

```



```
return dx
```

A.4 Caso a due strategie

A.4.1 Riduzione a una variabile

Nel caso a due strategie basta una sola variabile:

$$x_1 = x, \quad x_2 = 1 - x.$$

Ma, per chiarezza computazionale, e' utile mantenere anche la forma vettoriale

```
x = [x1, x2]
```

con vincolo

```
x1 + x2 = 1
```

A.4.2 Campo dinamico a una variabile

Se vuoi lavorare direttamente su x , puoi scrivere:

```
def two_strategy_payoffs(A, x):  
    f1 = A[0][0] * x + A[0][1] * (1.0 - x)  
    f2 = A[1][0] * x + A[1][1] * (1.0 - x)  
    return f1, f2
```

```
def two_strategy_field(A, x):  
    f1, f2 = two_strategy_payoffs(A, x)  
    return x * (1.0 - x) * (f1 - f2)
```

Questa e' la forma piu' comoda per l'analisi di linea di fase.

A.4.3 Integrazione numerica semplice

Per simulare la dinamica, si puo' usare Euler esplicito:

$$x_{t+1} = x_t + \Delta t, \dot{x}_t.$$

In Python:

```
def simulate_two_strategy_replicator(A, x0, dt, T):  
    x = x0  
    history_x = [x]
```

```

for t in range(T):
    dx = two_strategy_field(A, x)
    x = x + dt * dx

    if x < 0.0:
        x = 0.0
    if x > 1.0:
        x = 1.0

    history_x.append(x)

return history_x

```

A.4.4 Grafico della traiettoria

```

def plot_two_strategy_history(history_x):
    times = list(range(len(history_x)))

    plt.plot(times, history_x)
    plt.xlabel("tempo")
    plt.ylabel("x")
    plt.title("Dinamica replicativa a due strategie")
    plt.ylim(0.0, 1.0)
    plt.show()

```

Esempio:

```

A = [
    [3.0, 1.0],
    [2.0, 2.0]
]

history_x = simulate_two_strategy_replicator(
    A=A,
    x0=0.2,
    dt=0.05,
    T=300
)

plot_two_strategy_history(history_x)

```

A.4.5 Linea di fase

Per visualizzare il campo $\dot{x}$ sull'intervallo $[0, 1]$:

```

def plot_two_strategy_field(A, num_points=200):
    x_values = []
    dx_values = []

    for n in range(num_points + 1):
        x = n / num_points
        dx = two_strategy_field(A, x)

        x_values.append(x)
        dx_values.append(dx)

    plt.plot(x_values, dx_values)
    plt.xlabel("x")
    plt.ylabel("dx/dt")
    plt.title("Campo dinamico a due strategie")
    plt.axhline(0.0)
    plt.show()

```

Questo grafico e' molto utile per trovare e classificare qualitativamente i punti fissi.

A.5 Caso a tre strategie

A.5.1 Stato della popolazione

Nel caso a tre strategie lo stato e' un vettore

$x = [x_1, x_2, x_3]$

con

$x_1 + x_2 + x_3 = 1$

e tutte le componenti non negative.

A.5.2 Un passo di Euler nel caso generale

```

def euler_step_replicator(A, x, dt):
    dx = replicator_field(A, x)

    new_x = []
    for i in range(len(x)):
        new_x.append(x[i] + dt * dx[i])

    return new_x

```

A.5.3 Rinormalizzazione

Per evitare piccoli errori numerici, e' utile rinormalizzare il vettore:

```
def normalize_frequencies(x):
    clipped = []

    for value in x:
        if value < 0.0:
            clipped.append(0.0)
        else:
            clipped.append(value)

    total = sum(clipped)

    if total == 0.0:
        n = len(x)
        return [1.0 / n for _ in range(n)]

    normalized = []
    for value in clipped:
        normalized.append(value / total)

    return normalized
```

A.5.4 Simulazione completa per tre strategie

```
def simulate_replicator_general(A, x0, dt, T):
    x = x0[:]
    x = normalize_frequencies(x)

    history = [x[:]]

    for t in range(T):
        x = euler_step_replicator(A, x, dt)
        x = normalize_frequencies(x)
        history.append(x[:])

    return history
```

Esempio:

```
A = [
    [0.0, 2.0, 1.0],
    [1.0, 0.0, 2.0],
    [2.0, 1.0, 0.0]
]
```

```

history = simulate_replicator_general(
    A=A,
    x0=[0.2, 0.5, 0.3],
    dt=0.02,
    T=500
)

```

A.5.5 Grafico delle componenti nel tempo

```

def plot_three_strategy_history(history):
    times = list(range(len(history)))

    x1_values = []
    x2_values = []
    x3_values = []

    for x in history:
        x1_values.append(x[0])
        x2_values.append(x[1])
        x3_values.append(x[2])

    plt.plot(times, x1_values, label="x1")
    plt.plot(times, x2_values, label="x2")
    plt.plot(times, x3_values, label="x3")
    plt.xlabel("tempo")
    plt.ylabel("frequenza")
    plt.title("Dinamica replicativa a tre strategie")
    plt.ylim(0.0, 1.0)
    plt.legend()
    plt.show()

```

Questo non sostituisce un vero diagramma sul semplice, ma e' gia' molto utile didatticamente.

A.6 Mini-simulazione stocastica alla Moran

Questa parte e' molto importante, perche' collega la dinamica continua a una popolazione finita.

A.6.1 Idea del processo di Moran

Consideriamo una popolazione finita di dimensione N . Ogni individuo adotta una strategia. A ogni passo:

1. si calcola la fitness di ciascun tipo;
2. si sceglie un individuo da riprodurre con probabilita' proporzionale alla fitness;
3. si sceglie un individuo da rimpiazzare;
4. la popolazione cambia di una unita'.

Questa e' una versione elementare del meccanismo di Moran.

A.6.2 Popolazione a due strategie

Nel caso piu' semplice, basta tenere traccia del numero di individui del tipo 1. Se ci sono i individui di tipo 1 in una popolazione di taglia N , allora il numero di individui di tipo 2 e'

$$N - i.$$

La frequenza del tipo 1 e'

$$x = \frac{i}{N}.$$

A.6.3 Fitness media dei due tipi

Nel caso a due strategie con matrice

$$A = \begin{pmatrix} a_{11} & a_{12} & a_{21} & a_{22} \end{pmatrix},$$

una scelta semplice e' usare come fitness attesa:

$$f_1(x) = a_{11}x + a_{12}(1 - x),$$

$$f_2(x) = a_{21}x + a_{22}(1 - x).$$

In Python:

```
def moran_two_strategy_fitness(A, i, N):
    x = i / N

    f1 = A[0][0] * x + A[0][1] * (1.0 - x)
    f2 = A[1][0] * x + A[1][1] * (1.0 - x)

    return f1, f2
```

A.6.4 Un passo del processo di Moran

La logica del passo e' questa:

- la massa riproduttiva del tipo 1 e' proporzionale a if_1 ;
- quella del tipo 2 e' proporzionale a $(N-i)f_2$;
- poi si sceglie chi rimpiazzare uniformemente.

```
def moran_step_two_strategy(A, i, N):
    if i == 0:
        return 0
    if i == N:
        return N

    f1, f2 = moran_two_strategy_fitness(A, i, N)

    reproductive_mass_1 = i * f1
    reproductive_mass_2 = (N - i) * f2
    total_mass = reproductive_mass_1 + reproductive_mass_2

    if total_mass <= 0.0:
        prob_reproduce_1 = i / N
    else:
        prob_reproduce_1 = reproductive_mass_1 / total_mass

    u_birth = random.random()
    if u_birth < prob_reproduce_1:
        reproducing_type = 1
    else:
        reproducing_type = 2

    u_death = random.random()
    if u_death < i / N:
        removed_type = 1
    else:
        removed_type = 2

    if reproducing_type == 1 and removed_type == 2:
        i = i + 1
    elif reproducing_type == 2 and removed_type == 1:
        i = i - 1

    return i
```

Questa e' la mini-simulazione Moran piu' semplice che abbia ancora un chiaro significato evolutivo.

A.6.5 Simulazione completa alla Moran

```
def simulate_moran_two_strategy(A, i0, N, T):
    i = i0
    history_i = [i]
    history_x = [i / N]

    for t in range(T):
        i = moran_step_two_strategy(A, i, N)
        history_i.append(i)
        history_x.append(i / N)

    results = {
        "history_i": history_i,
        "history_x": history_x
    }

    return results
```

Esempio:

```
A = [
    [3.0, 1.0],
    [2.0, 2.0]
]

results_moran = simulate_moran_two_strategy(
    A=A,
    i0=20,
    N=100,
    T=500
)
```

A.6.6 Grafico della traiettoria Moran

```
def plot_moran_history(history_x):
    times = list(range(len(history_x)))

    plt.plot(times, history_x)
    plt.xlabel("tempo")
    plt.ylabel("frequenza del tipo 1")
    plt.title("Processo di Moran a due strategie")
    plt.ylim(0.0, 1.0)
    plt.show()
```

Esempio:


```
plot_moran_history(results_moran["history_x"])
```

A.6.7 Ripetere molte simulazioni Moran

Per confrontare il modello finito con la dinamica replicativa continua, conviene ripetere molte realizzazioni indipendenti.

```
def run_many_moran_simulations(A, i0, N, T, num_runs):
    all_histories = []

    for run in range(num_runs):
        results = simulate_moran_two_strategy(A, i0, N, T)
        all_histories.append(results["history_x"])

    return all_histories
```

A.6.8 Media empirica di molte traiettorie

```
def average_histories(histories):
    T = len(histories[0])
    mean_history = []

    for t in range(T):
        values_t = []
        for history in histories:
            values_t.append(history[t])

        mean_history.append(statistics.mean(values_t))

    return mean_history
```

A.6.9 Confronto tra Moran e replicator

```
def compare_replicator_and_moran(A, x0, N, T, dt, num_runs):
    history_rep = simulate_two_strategy_replicator(
        A=A,
        x0=x0,
        dt=dt,
        T=T
    )

    i0 = int(round(x0 * N))

    histories_moran = run_many_moran_simulations(
```

```

        A=A,
        i0=i0,
        N=N,
        T=T,
        num_runs=num_runs
    )

    mean_moran = average_histories(histories_moran)

    plt.plot(range(len(history_rep)), history_rep, label="replicator")
    plt.plot(range(len(mean_moran)), mean_moran, label="media Moran")
    plt.xlabel("tempo")
    plt.ylabel("frequenza del tipo 1")
    plt.title("Confronto tra dinamica continua e Moran")
    plt.ylim(0.0, 1.0)
    plt.legend()
    plt.show()

```

Esempio:

```

A = [
    [3.0, 1.0],
    [2.0, 2.0]
]

```

```

compare_replicator_and_moran(
    A=A,
    x0=0.2,
    N=100,
    T=300,
    dt=0.05,
    num_runs=50
)

```

Qui bisogna fare attenzione a una cosa concettuale: il tempo della replicator equation e quello del processo di Moran non coincidono automaticamente. Tuttavia, per una mini-simulazione didattica il confronto qualitativo e' gia' molto istruttivo.

A.7 Organizzazione consigliata del file

Per mantenere il codice leggibile, conviene organizzarlo cosi':

1. import delle librerie;
2. funzioni di base:
 - `matrix_vector_product`

- average_fitness
- replicator_field

3. caso a due strategie:

- two_strategy_payoffs
- two_strategy_field
- simulate_two_strategy_replicator
- grafici

4. caso generale:

- euler_step_replicator
- normalize_frequencies
- simulate_replicator_general

5. Moran:

- moran_two_strategy_fitness
- moran_step_two_strategy
- simulate_moran_two_strategy
- simulazioni multiple

6. blocco finale con esempi.

Per esempio:

```
if __name__ == "__main__":
    A = [
        [3.0, 1.0],
        [2.0, 2.0]
    ]

    history_rep = simulate_two_strategy_replicator(
        A=A,
        x0=0.2,
        dt=0.05,
        T=300
    )
    plot_two_strategy_history(history_rep)

    results_moran = simulate_moran_two_strategy(
        A=A,
        i0=20,
        N=100,
        T=300
    )
    plot_moran_history(results_moran["history_x"])

    compare_replicator_and_moran(
```

```

A=A,
x0=0.2,
N=100,
T=300,
dt=0.05,
num_runs=50
)

```

A.8 Perché questa appendice è utile

Questa appendice è particolarmente importante perché rende visibile una progressione metodologica molto bella:

1. prima si costruisce la dinamica deterministica delle frequenze;
2. poi la si integra numericamente;
3. infine la si confronta con una dinamica discreta e stocastica a popolazione finita.

Questo è uno dei punti più forti dell'intera dispensa.

A.9 Conclusione dell'appendice

La struttura proposta qui è volutamente semplice. Chi conosce Python può implementarla quasi direttamente; chi usa altri linguaggi può leggerla come pseudocodice molto vicino a una traduzione operativa.

Il messaggio metodologico è che le dinamiche replicative offrono un esempio eccellente di collegamento tra:

- modello continuo;
- simulazione numerica;
- popolazione finita;
- rumore demografico;
- confronto tra traiettoria media e realizzazioni individuali.